

Mobile App Error Handling and Troubleshooting

Mobile identity, network, payment and release troubleshooting.



Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository:
rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @
3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Mobile identity, network, payment and release troubleshooting. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

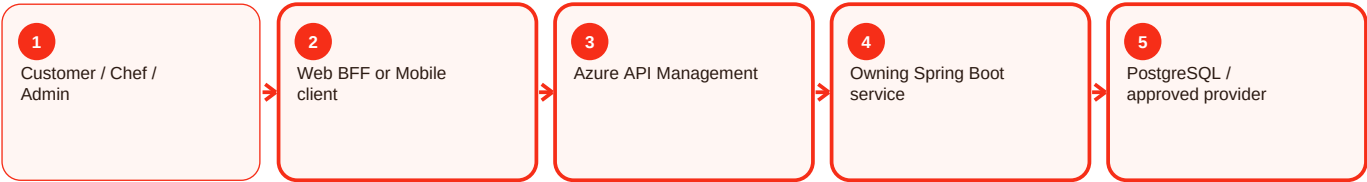
A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

Mobile evidence boundary - Overview and responsibility

Foundation / partial

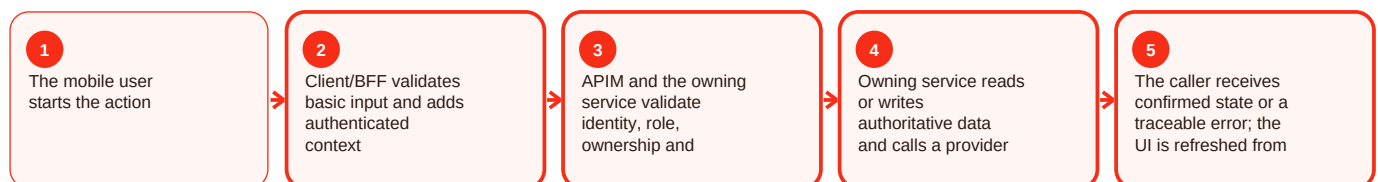
Summary

Mobile evidence boundary is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile evidence boundary, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence:
 docs/handover/2026-07-30-customer-mobile-auth-foundation.md;
 docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile evidence boundary - Functional behavior and data

Foundation / partial

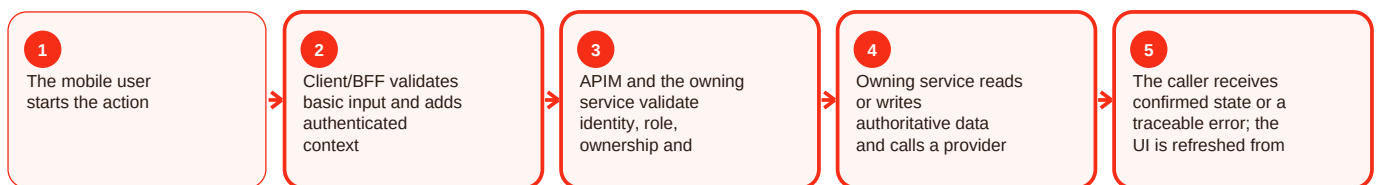
Summary

Mobile evidence boundary is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile evidence boundary, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence:
 docs/handover/2026-07-30-customer-mobile-auth-foundation.md;
 docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile evidence boundary - Operations, errors and deployment

Foundation / partial

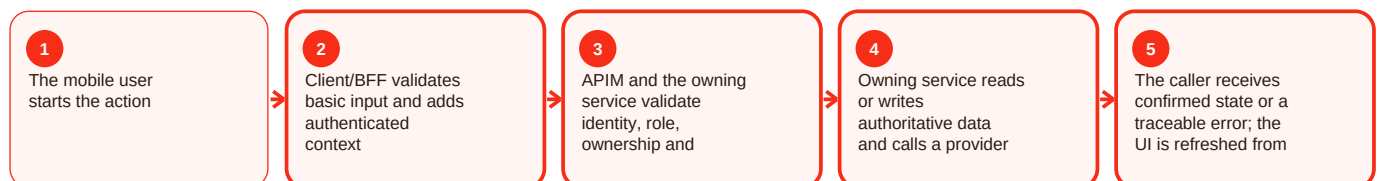
Summary

Mobile evidence boundary is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile evidence boundary, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Entra external id - Overview and responsibility

Foundation / partial

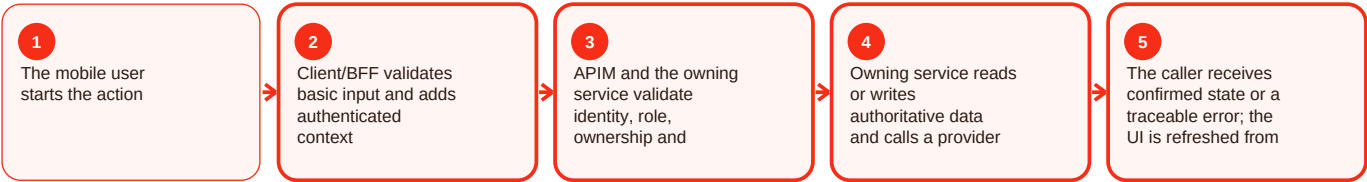
Summary

Entra external id is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Entra External ID, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Entra external id - Functional behavior and data

Foundation / partial

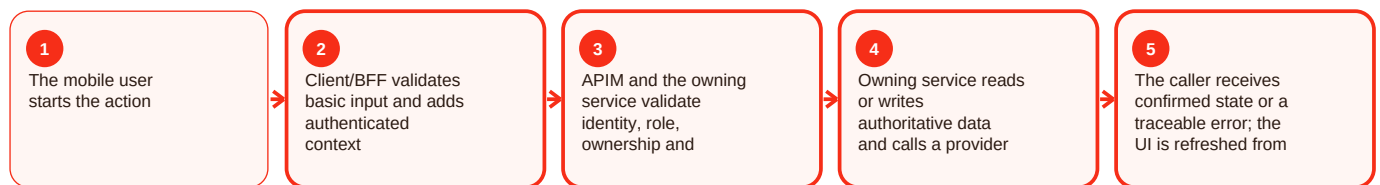
Summary

Entra external id is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Entra External ID, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Entra external id - Operations, errors and deployment

Current implementation

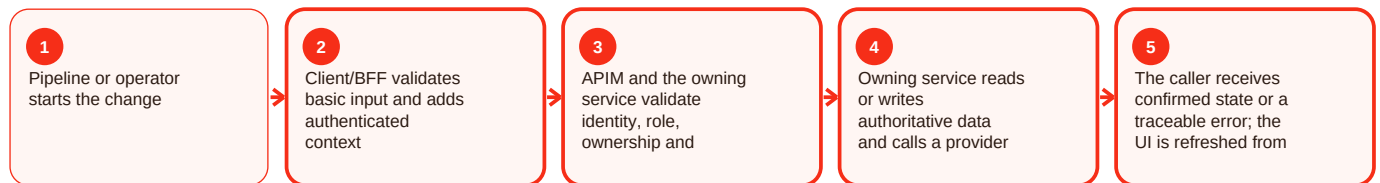
Summary

Entra external id is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For Entra External ID, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pkce - Overview and responsibility

Foundation / partial

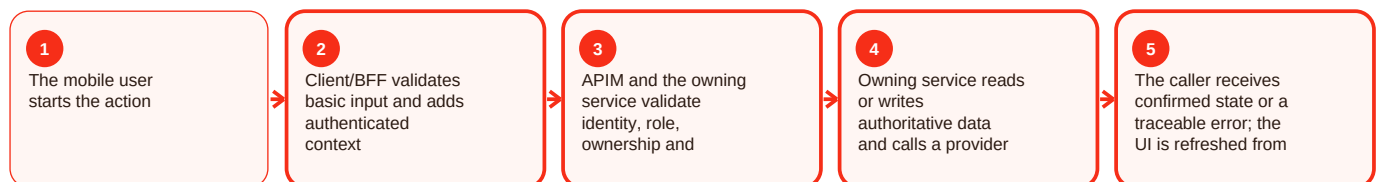
Summary

Pkce is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For PKCE, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pkce - Functional behavior and data

Foundation / partial

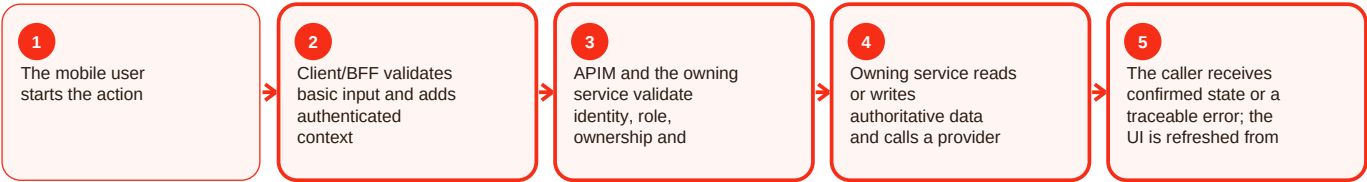
Summary

Pkce is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For PKCE, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pkce - Operations, errors and deployment

Foundation / partial

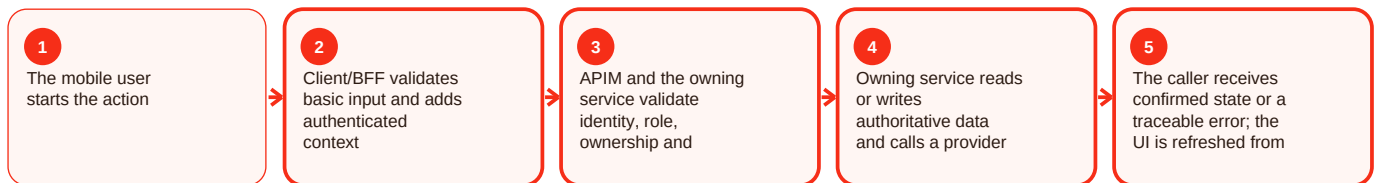
Summary

Pkce is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For PKCE, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mfa - Overview and responsibility

Foundation / partial

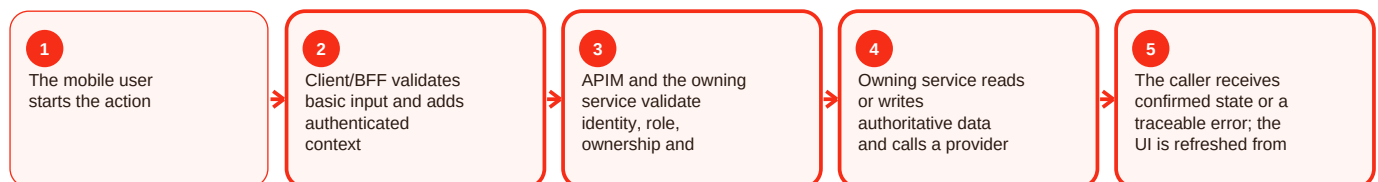
Summary

Mfa is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For MFA, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mfa - Functional behavior and data

Foundation / partial

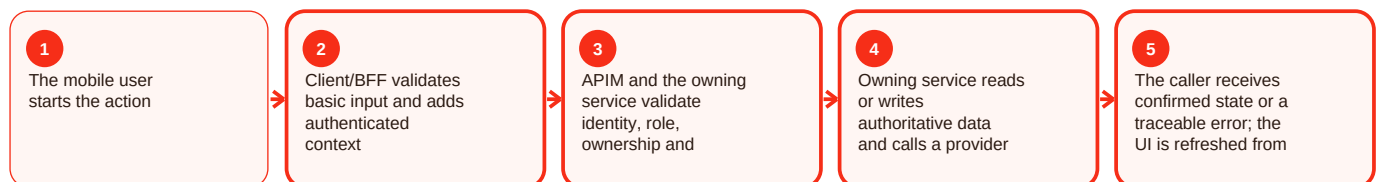
Summary

Mfa is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For MFA, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mfa - Operations, errors and deployment

Current implementation

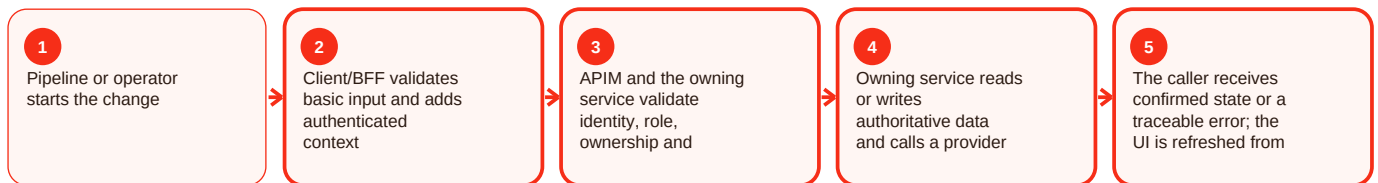
Summary

Mfa is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For MFA, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Interactive sign-in - Overview and responsibility

Current implementation

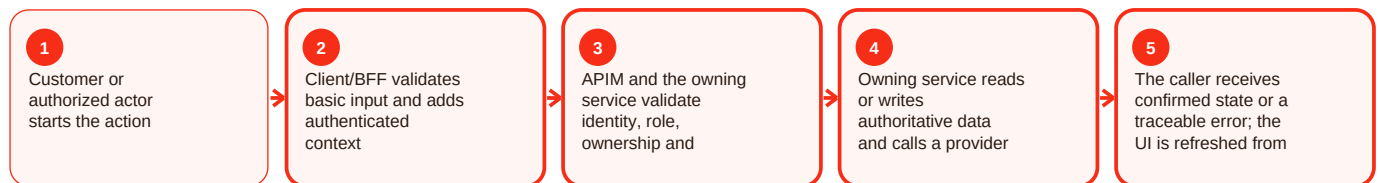
Summary

Interactive sign-in is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For interactive sign-in, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Interactive sign-in - Functional behavior and data

Current implementation

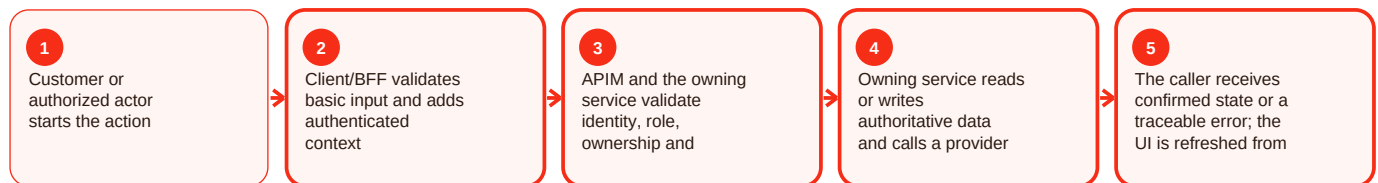
Summary

Interactive sign-in is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For interactive sign-in, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Interactive sign-in - Operations, errors and deployment

Current implementation

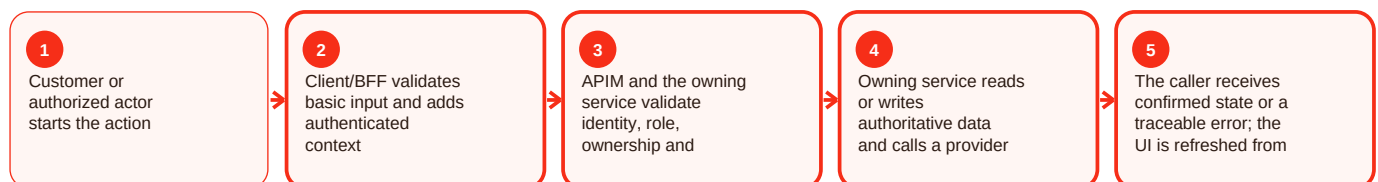
Summary

Interactive sign-in is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For interactive sign-in, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refresh token secure storage - Overview and responsibility

Current implementation

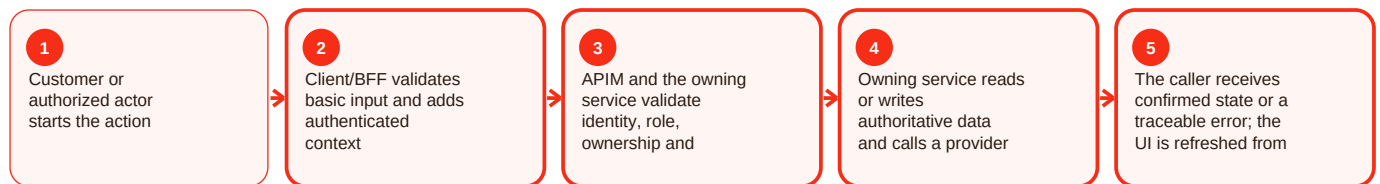
Summary

Refresh token secure storage is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For refresh token secure storage, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refresh token secure storage - Functional behavior and data

Current implementation

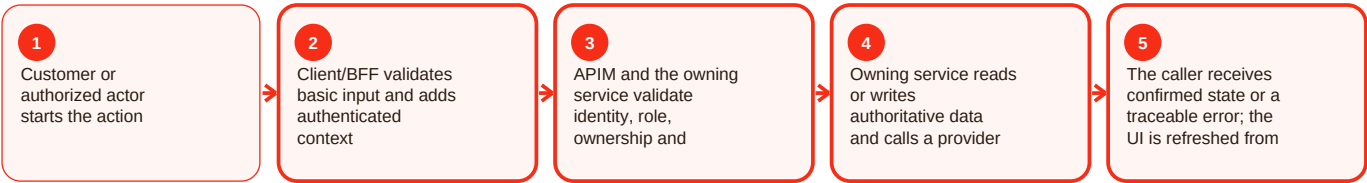
Summary

Refresh token secure storage is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For refresh token secure storage, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refresh token secure storage - Operations, errors and deployment

Current implementation

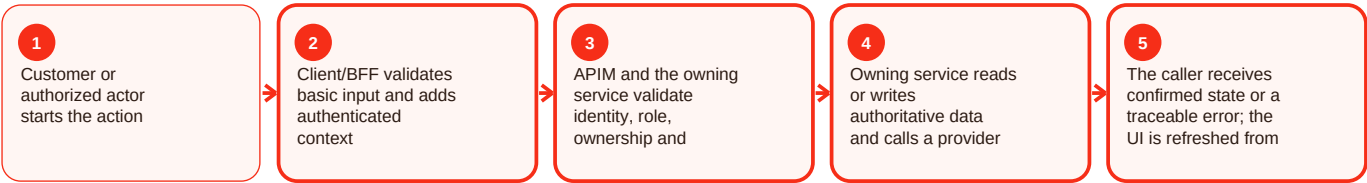
Summary

Refresh token secure storage is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For refresh token secure storage, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Access token memory - Overview and responsibility

Current implementation

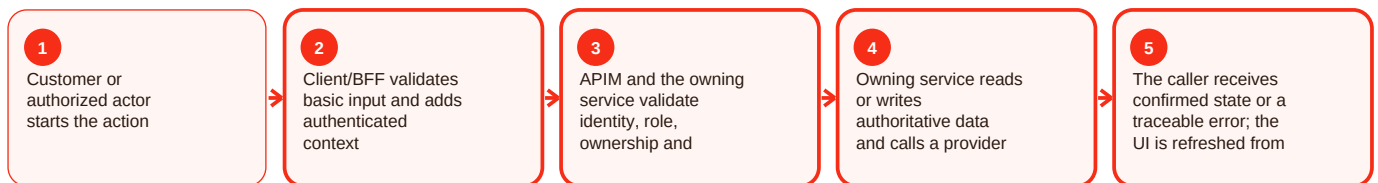
Summary

Access token memory is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For access token memory, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Access token memory - Functional behavior and data

Current implementation

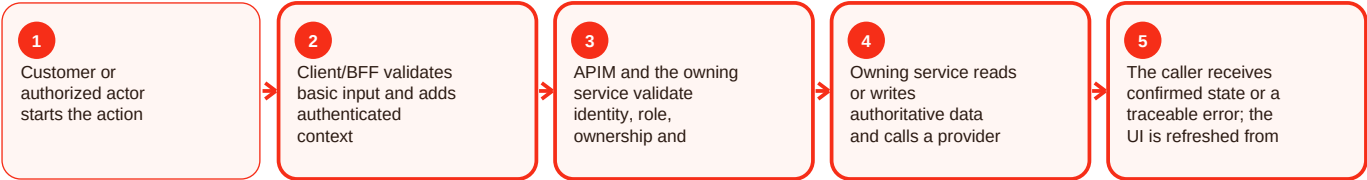
Summary

Access token memory is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For access token memory, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Access token memory - Operations, errors and deployment

Current implementation

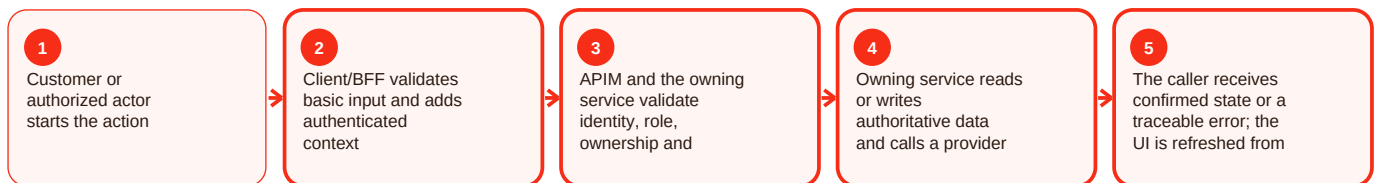
Summary

Access token memory is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For access token memory, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim bearer calls - Overview and responsibility

Current implementation

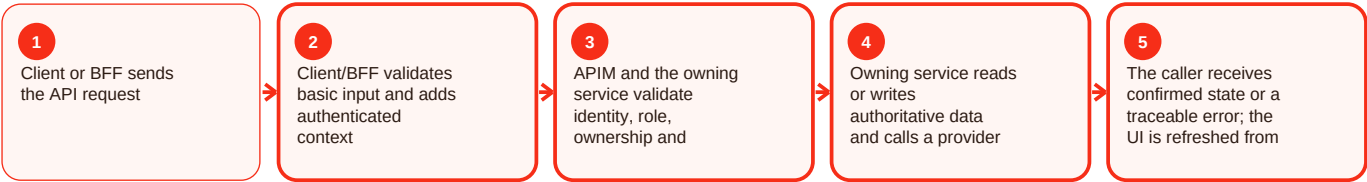
Summary

Apim bearer calls is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM bearer calls, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim bearer calls - Functional behavior and data

Current implementation

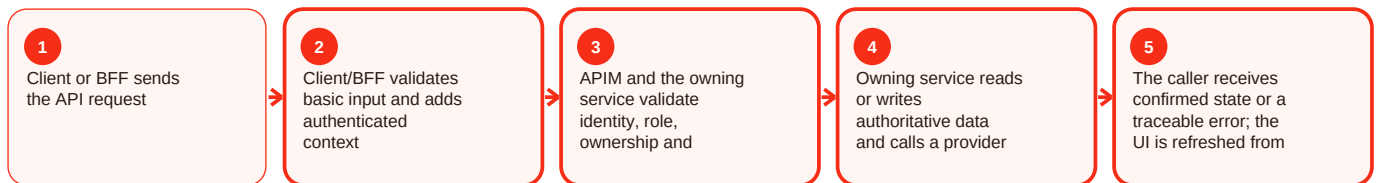
Summary

Apim bearer calls is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM bearer calls, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim bearer calls - Operations, errors and deployment

Current implementation

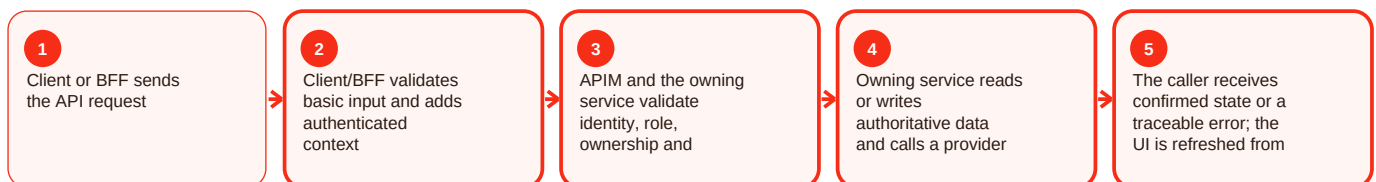
Summary

Apim bearer calls is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM bearer calls, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

401 refresh / retry - Overview and responsibility

Current implementation

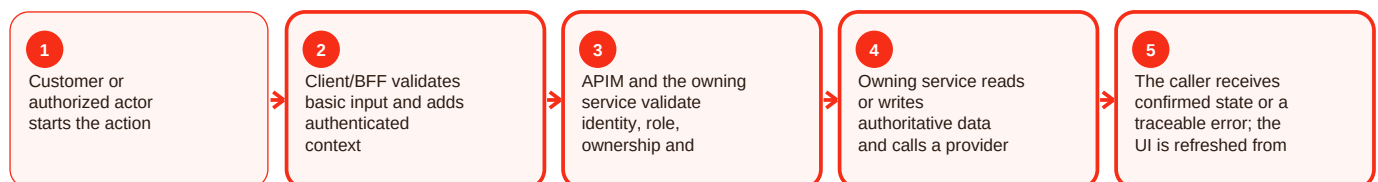
Summary

401 refresh / retry is part of the errors area of Craves. A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence. For 401 refresh/retry, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 Unauthorized: Authentication is missing/expired/invalid. Resolution: Use supported identity/refresh flow and capture request ID.

5xx/upstream unavailable: Service/database/provider/deployment is unhealthy. Resolution: Use correlation evidence to isolate the boundary and rollback if release-related.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

401 refresh / retry - Functional behavior and data

Current implementation

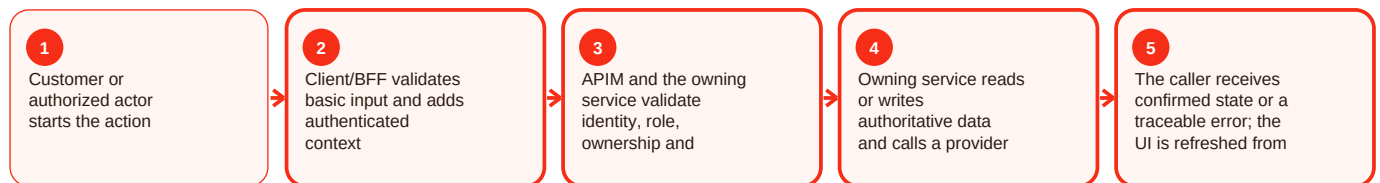
Summary

401 refresh / retry is part of the errors area of Craves. A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence. For 401 refresh/retry, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 Unauthorized: Authentication is missing/expired/invalid. Resolution: Use supported identity/refresh flow and capture request ID.

5xx/upstream unavailable: Service/database/provider/deployment is unhealthy. Resolution: Use correlation evidence to isolate the boundary and rollback if release-related.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

401 refresh / retry - Operations, errors and deployment

Current implementation

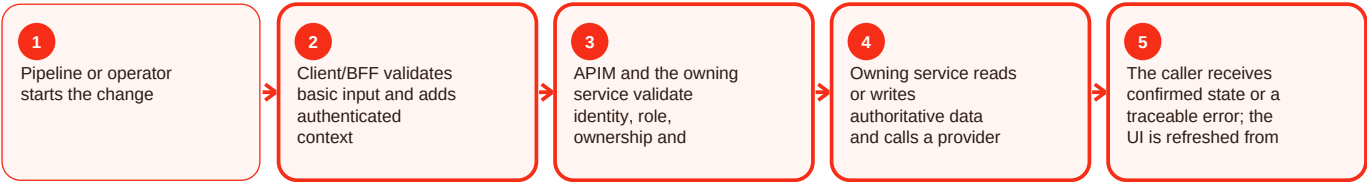
Summary

401 refresh / retry is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For 401 refresh/retry, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Logout - Overview and responsibility

Foundation / partial

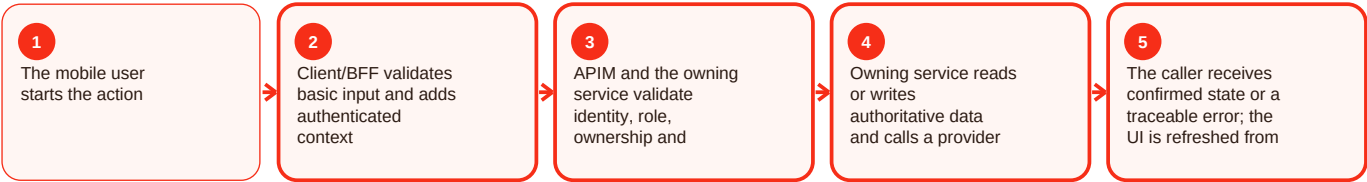
Summary

Logout is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For logout, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Logout - Functional behavior and data

Foundation / partial

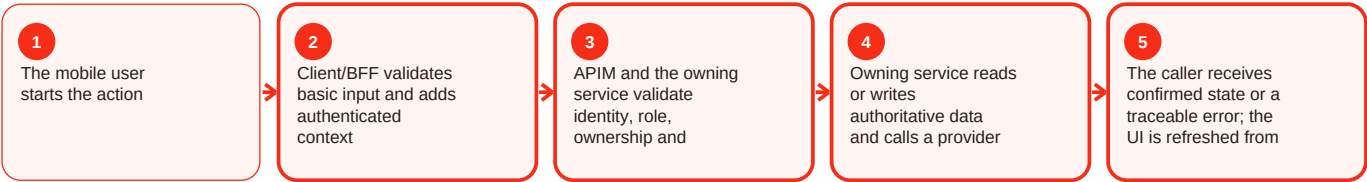
Summary

Logout is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For logout, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Logout - Operations, errors and deployment

Current implementation

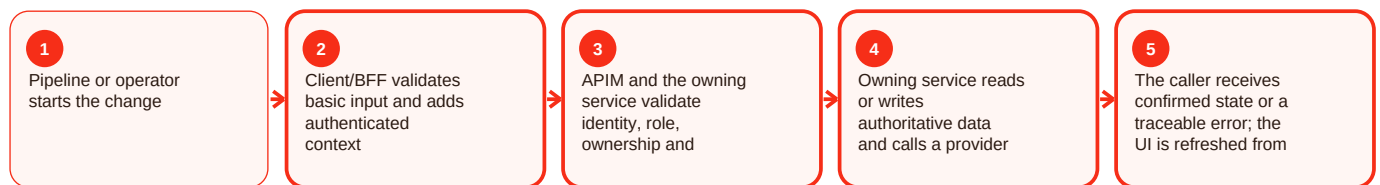
Summary

Logout is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For logout, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Environment config - Overview and responsibility

Current implementation

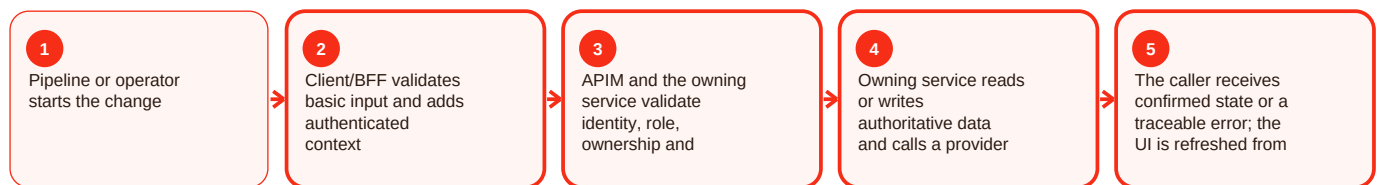
Summary

Environment config is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For environment config, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Environment config - Functional behavior and data

Current implementation

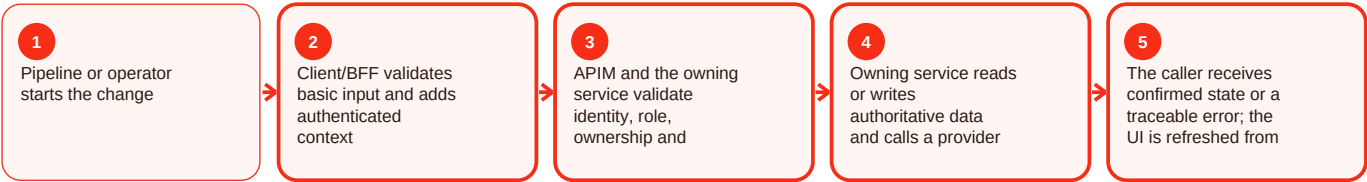
Summary

Environment config is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For environment config, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Environment config - Operations, errors and deployment

Current implementation

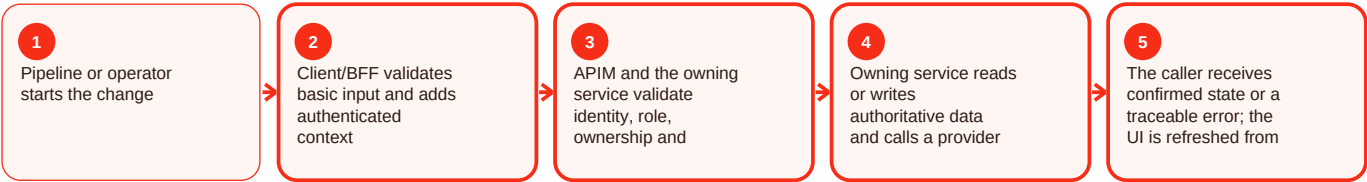
Summary

Environment config is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For environment config, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Keychain - Overview and responsibility

Foundation / partial

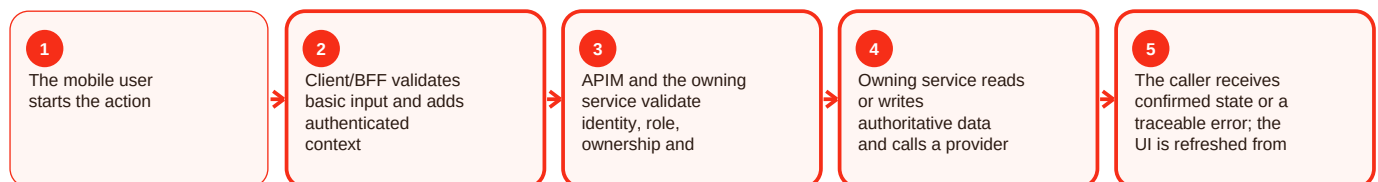
Summary

Keychain is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Keychain, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Keychain - Functional behavior and data

Foundation / partial

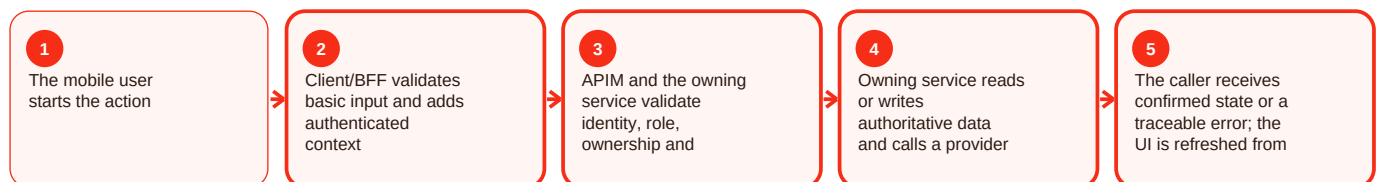
Summary

Keychain is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Keychain, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Keychain - Operations, errors and deployment

Foundation / partial

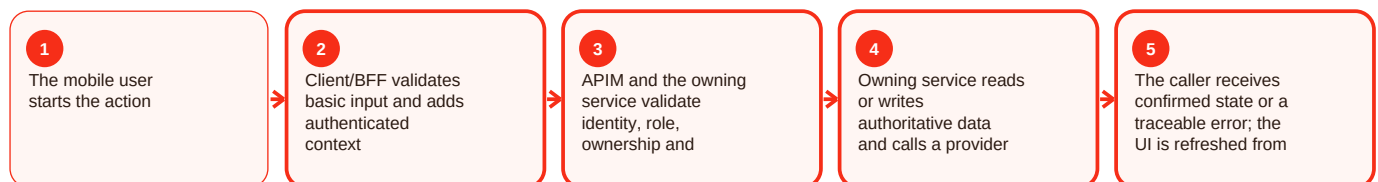
Summary

Keychain is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Keychain, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Android keystore - Overview and responsibility

Foundation / partial

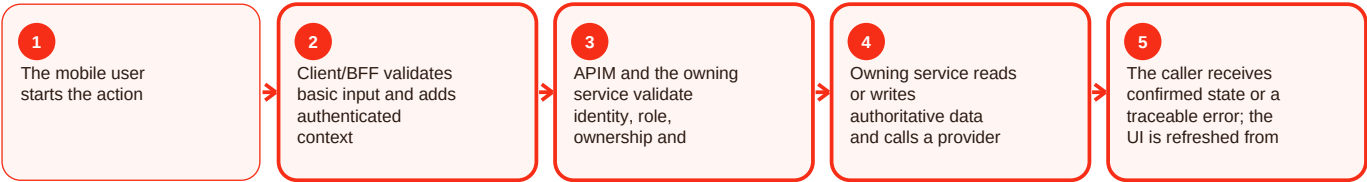
Summary

Android keystore is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Android Keystore, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Android keystore - Functional behavior and data

Foundation / partial

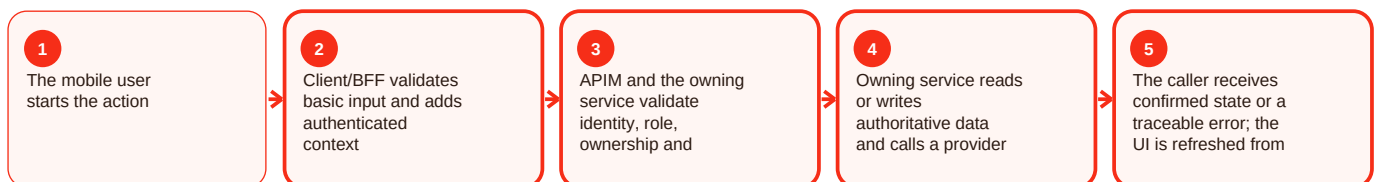
Summary

Android keystore is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Android Keystore, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Android keystore - Operations, errors and deployment

Foundation / partial

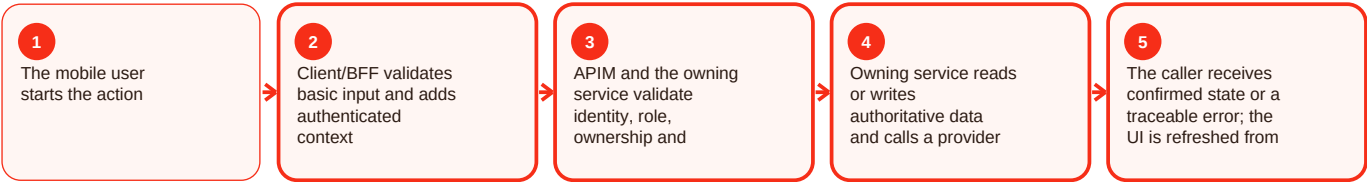
Summary

Android keystore is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For Android Keystore, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence:
docs/handover/2026-07-30-customer-mobile-auth-foundation.md;
docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Network / offline - Overview and responsibility

Foundation / partial

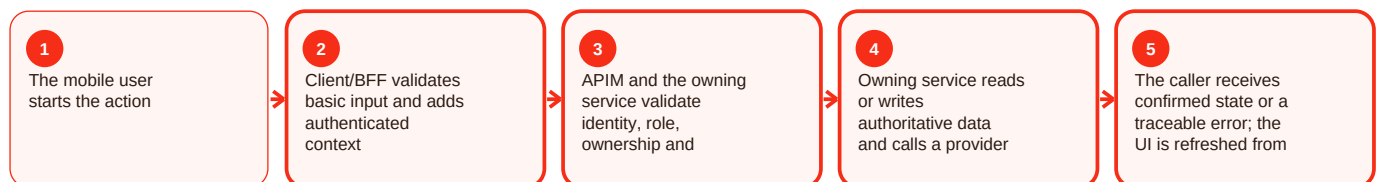
Summary

Network / offline is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For network/offline, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Network / offline - Functional behavior and data

Foundation / partial

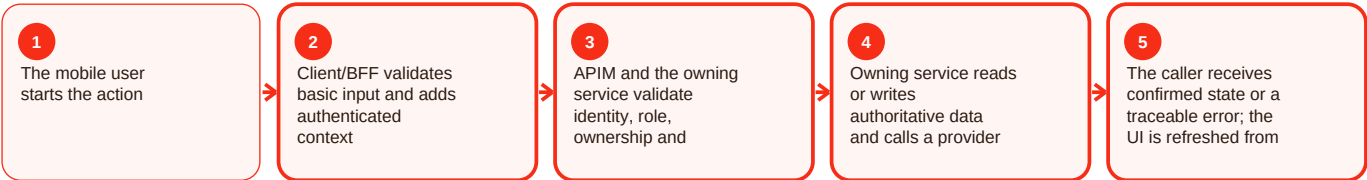
Summary

Network / offline is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For network/offline, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence:
 docs/handover/2026-07-30-customer-mobile-auth-foundation.md;
 docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Network / offline - Operations, errors and deployment

Current implementation

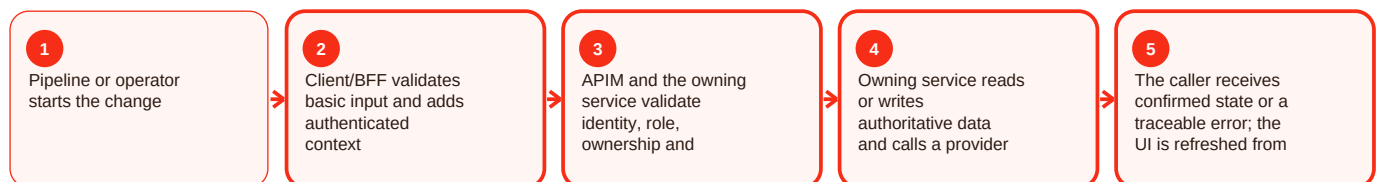
Summary

Network / offline is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For network/offline, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Profile / discovery contracts - Overview and responsibility

Current implementation

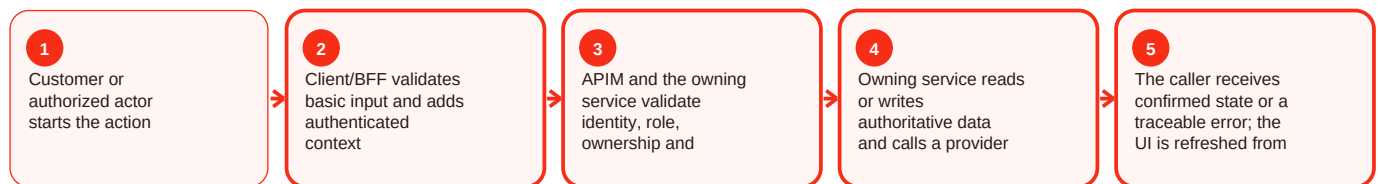
Summary

Profile / discovery contracts is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For profile/discovery contracts, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius.

Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution:

Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence.

Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Profile / discovery contracts - Functional behavior and data

Current implementation

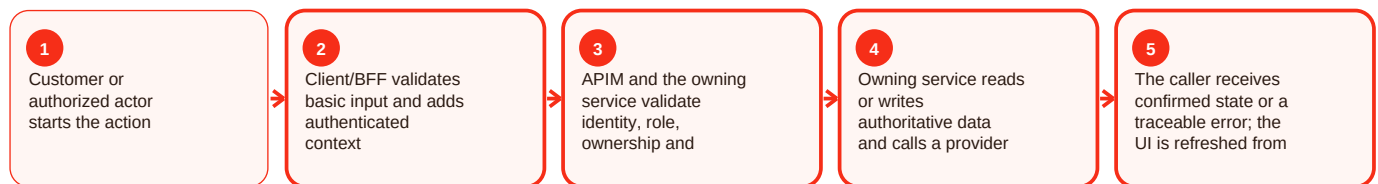
Summary

Profile / discovery contracts is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For profile/discovery contracts, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius.

Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution:

Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence.

Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Profile / discovery contracts - Operations, errors and deployment

Current implementation

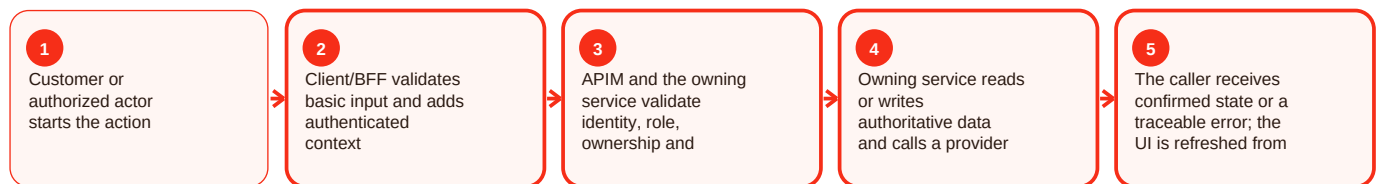
Summary

Profile / discovery contracts is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For profile/discovery contracts, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius.

Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution:

Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence.

Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart / checkout contract - Overview and responsibility

Current implementation

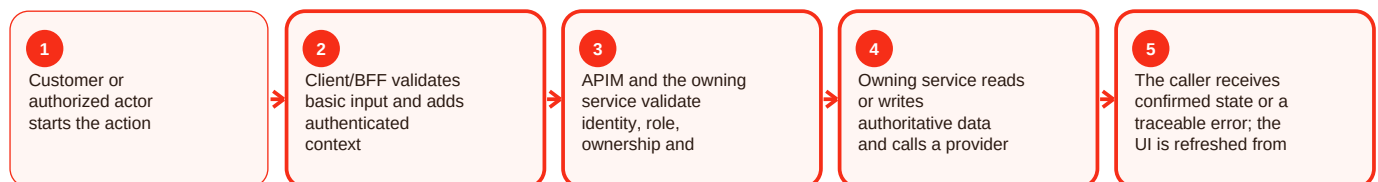
Summary

Cart / checkout contract is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart/checkout contract, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart / checkout contract - Functional behavior and data

Current implementation

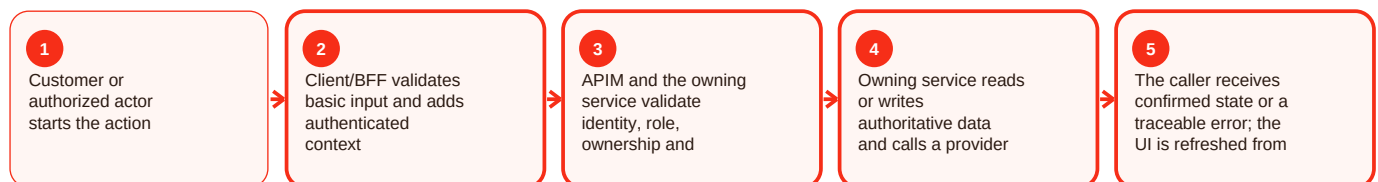
Summary

Cart / checkout contract is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart/checkout contract, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart / checkout contract - Operations, errors and deployment

Current implementation

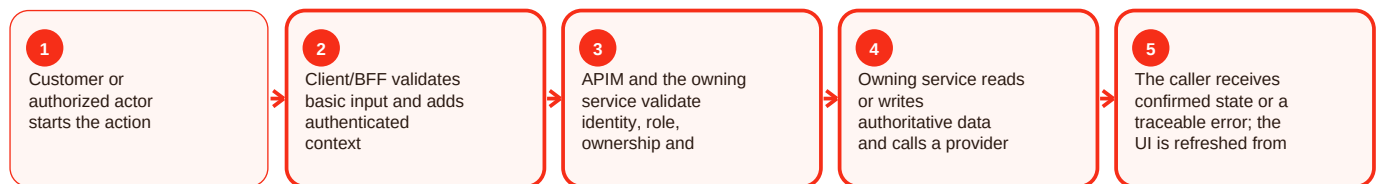
Summary

Cart / checkout contract is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart/checkout contract, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Hosted payment handoff - Overview and responsibility

Current implementation

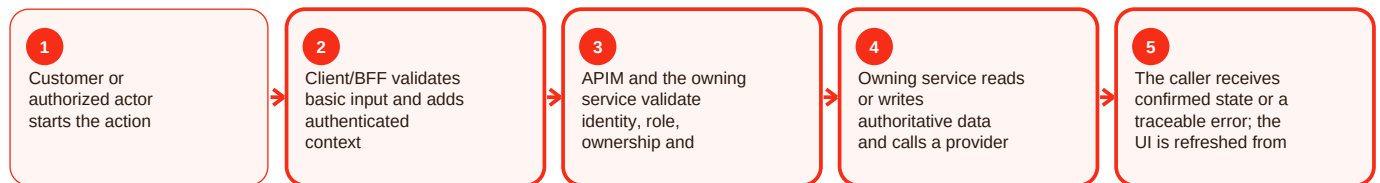
Summary

Hosted payment handoff is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For hosted payment handoff, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Hosted payment handoff - Functional behavior and data

Current implementation

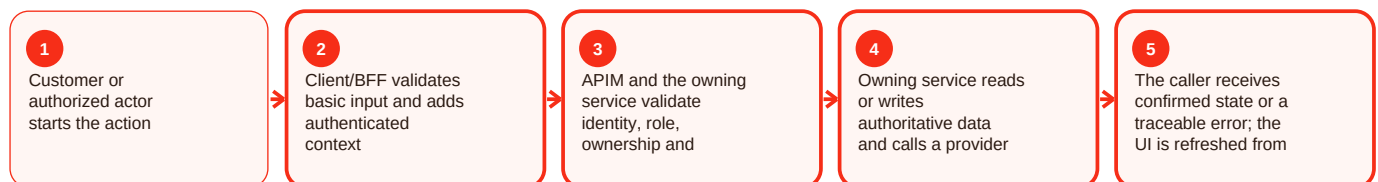
Summary

Hosted payment handoff is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For hosted payment handoff, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Hosted payment handoff - Operations, errors and deployment

Current implementation

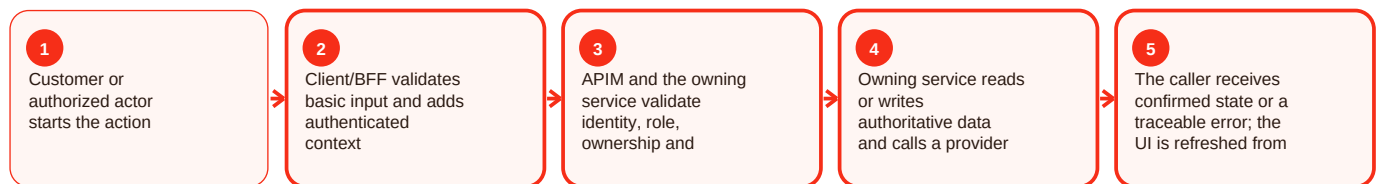
Summary

Hosted payment handoff is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For hosted payment handoff, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Orders / tracking - Overview and responsibility

Current implementation

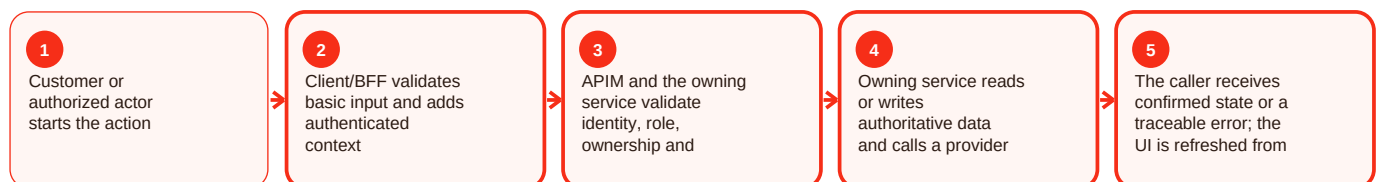
Summary

Orders / tracking is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For orders/tracking, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.

Repository gaps that must stay visible

- APIM README cites `infra/apim/craves-openapi.yaml`, but that literal artifact was not resolvable on main during this evidence snapshot.
- `customer-web-next` README names `azure-pipelines-customer-web-next.yml`, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.